Facultad de Ingeniería-UNMDP

Calidad de software “A”

TRABAJO PRÁCTICO GRUPAL

Fecha de entrega: 11/11/2026

Docentes a cargo:

Adolfo Spinelli, Jessica Elizabeth Miguel

y Paula Bonifazi Aquino

Integrantes:

Santiago Soldani, Iván Etcheverry, Marco Ioli
y Jeremias Arango
(grupo 8)

Introducción

En este documento vamos a estar detallando cómo fue el proceso de Testing de un producto de Software. Para llevar a cabo esto, pasamos por 3 etapas. La primera es el testing de Caja Negra, donde nos enfocaremos en las pruebas unitarias, o atómicas, sobre la clase “AsociadoDTO”. Luego, realizamos pruebas de Integración, testeando el trabajo en conjunto entre diferentes componentes, agregando la base de Datos. Por último trabajamos con unas pruebas de integración con Mocks.
En los casos que se consideraron necesarios, las pruebas fueron planificadas con una planilla de cálculo, la cuál será adjuntada con este documento, a modo complemento.

Testing de Caja Negra

Para esta primera consigna del enunciado, el objetivo de esta etapa fue desarrollar una suite de pruebas unitarias sobre la clase “AsociadoDTO” de forma estrictamente aislada, evaluando la correcta asignación, recuperación y consistencia de los datos sin interactuar con componentes externos.

Al trabajar sobre Testing de Caja Negra, el diseño de las pruebas se basó exclusivamente en la especificación formal del componente provista en el SRS, y en los contratos públicos de los métodos de la clase a probar.

Para sistematizar la cobertura, nos apoyamos en una planilla de cálculo complementaria al informe, donde se aplicaron las técnicas de Partición en Clases de Equivalencia y generamos una batería de pruebas sobre los parámetros para lograr el mayor porcentaje de cobertura.

Algunos de los criterios que adoptamos al momento del diseño de las pruebas fueron los siguientes:

- Exclusión de pruebas fuera de precondición:

En la especificación formal (SRS),se establece de manera explícita para los atributos de texto (nombre, apellido, dni, ciudad, calle, teléfono) la regla: "*No puede ser nulo*". Las precondiciones delimitan estrictamente el dominio de entradas válidas bajo el cual el método garantiza cumplir su postcondición. Si el cliente invocador pasa un valor null, el comportamiento posterior queda fuera del alcance de la garantía funcional. Por esta razón, decidimos no diseñar casos de prueba con valores nulos, ya que en caja negra no corresponde probar entradas que caen fuera de las precondiciones acordadas.

- Tratamiento de la Cadena Vacía ("") como entrada admitida:

Al contrastar el código con el SRS, observamos que la especificación únicamente exige que las cadenas no sean nulas, pero no define restricciones de longitud mínima ni formatos para nombres o direcciones. Por ende, desde la rigurosidad de la caja negra, la cadena vacía clasifica formalmente como un valor de prueba perteneciente a la clase válida, a pesar de que desde el punto de vista del negocio carece de sentido que un asociado pueda registrarse sin caracteres en su nombre o DNI.

- Verificación simétrica Setter / Getter y Constructor:

Dado que los métodos de AsociadoDTO consisten fundamentalmente en pares de acceso y mutación, los casos sobre los setters se validaron comprobando la consistencia mediante el getter correspondiente. Adicionalmente, se incluyó la verificación del constructor parametrizado para constatar que el mapeo inicial de todos los atributos respete íntegramente los valores provistos.

- Aislamiento de la restricción de unicidad del DNI:

El SRS indica que el DNI "actúa como identificador único". Sin embargo, al tratarse de pruebas unitarias y atómicas sobre AsociadoDTO, el componente no gestiona colecciones en memoria ni almacenamiento persistente. En consecuencia, implementamos una prueba (testDniDuplicado) para comprobar que dos instancias independientes pueden retener el mismo valor de DNI sin arrojar errores a nivel de objeto, constatando que la responsabilidad de aplicar y validar la unicidad (clave primaria o restricción UNIQUE) no pertenece al DTO, sino que debe delegarse y verificarse en la capa de persistencia.

- Estructuración del escenario con JUnit (@Before / setUp):

Para asegurar la independencia y repetibilidad de cada caso de prueba, adoptamos la anotación @Before. En lugar de instanciar manualmente el objeto en cada método, el entorno se reinicializa antes de cada ejecución, garantizando que el estado previo no condicione los resultados de las pruebas subsiguientes.

Descripción Detallada de los Métodos de la Suite de Caja Negra

A continuación se detalla qué evalúa cada método de prueba diseñado en Test_AsociadoDTO, especificando cómo se construyó el caso, qué operación ejecuta y qué resultado verifica sin necesidad de inspeccionar el código fuente:

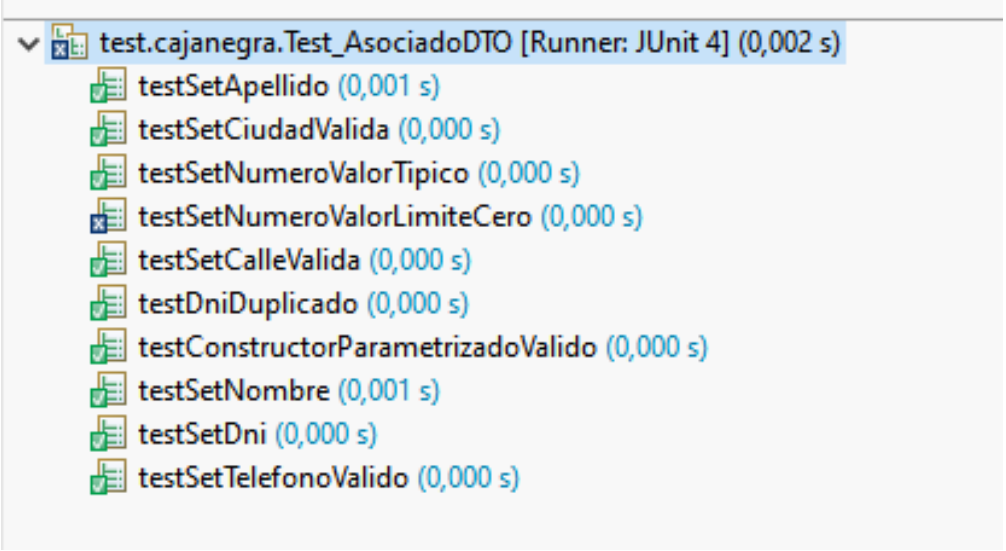
1. **Pruebas de Datos Personales e Identificación (nombre, apellido, dni)**
   - **testSetNombre**: Evalúa la correcta asignación y lectura del atributo nombre. Se invoca setNombre("Carlos") con una cadena representativa de la clase de equivalencia válida y se comprueba mediante assertEquals("Carlos", asociado.getNombre()) que el valor recuperado coincida exactamente con el suministrado.
   - **testSetApellido**: Verifica la mutación y consulta del campo apellido. Se envía el valor típico "Perez" a setApellido("Perez") y se valida con assertEquals("Perez", asociado.getApellido()) que el estado interno retenga la cadena sin alteraciones.
   - **testSetDni**: Comprueba la asignación del documento nacional de identidad. Se ejecuta setDni("12345678") con un valor numérico estándar en formato texto y se verifica la coherencia del dato mediante assertEquals("12345678", asociado.getDni()).

2. **Pruebas de Domicilio, Contacto y Análisis de Valores Límite (calle, numero, ciudad, telefono)**
   - **testSetCalleValida**: Evalúa la asignación del nombre de la calle del domicilio. Se invoca setCalle("San Martin") sobre la instancia limpia provista por @Before y se corrobora mediante assertEquals que el método accesor devuelva la misma cadena.
   - **testSetCiudadValida**: Comprueba la carga de la localidad del asociado. Se envía "Mar del Plata" a setCiudad("Mar del Plata") y se verifica la correspondencia del campo mediante assertEquals con getCiudad().
   - **testSetTelefonoValido**: Verifica el registro del número de contacto telefónico. Se suministra la cadena numérica "2231234567" a setTelefono("2231234567") y se constata con assertEquals la igualdad frente a getTelefono().
   - **testSetNumeroValorTipico**: Evalúa la asignación de la altura catastral con un valor entero positivo típico. Se invoca setNumero(23) dentro de la clase de equivalencia válida y se confirma mediante assertEquals(23, asociado.getNumero()) que el número se almacena correctamente.
   - **testSetNumeroValorLimiteCero**: Somete al método setNumero a su valor frontera inferior válido según el documento SRS 3.1 (numero >= 0). Se invoca setNumero(0) esperando que la altura cero sea admitida para domicilios sin numeración formal, referencias de ruta o parcelas. Esta prueba falla en rojo al disparar un AssertionError inesperado, poniendo en evidencia el defecto de implementación del DTO frente al requisito formal.

3. **Pruebas de Integridad de Dominio y Construcción Completa**
   - **testDniDuplicado**: Evalúa el comportamiento del DTO frente a la regla de unicidad del SRS ("actúa como identificador único") en un contexto unitario y estrictamente aislado. Se instancian dos objetos independientes (socio1 y socio2) asignándoles a ambos el mismo DNI ("12345678"). Se comprueba mediante assertEquals(socio2.getDni(), socio1.getDni()) que ambas instancias

retienen el dato sin interferencias ni excepciones, demostrando que la responsabilidad de aplicar y garantizar la unicidad pertenece a la capa de persistencia (motor de base de datos) y no a la estructura en memoria del DTO.

- **testConstructorParametrizadoValido:** Verifica la inicialización integral de la entidad a través de su constructor sobrecargado. Se invoca new AsociadoDTO("Juan", "Perez", "12345678", "Mar del Plata", "San Martin", 1234, "2231234567") y se valida mediante aserciones individuales que cada uno de los siete atributos haya sido mapeado de forma consistente, ordenada y sin omisiones.



Suite de pruebas para la clase AsociadoDTO

En la imagen de arriba, se ven listados los métodos de tipo "*@Test*" que elegimos desarrollar para hacer las pruebas que consideramos suficientes. Vemos que desarrollamos pruebas sobre los métodos llamados 'setters', ya que estos a su vez requieren la corroboración de los resultados con los métodos 'getters' correspondientes.

Observación: Cabe aclarar, cómo se puede observar en la imagen, que para elegir los valores para las pruebas, identificamos a partir de la Especificación de Requerimientos (SRS), que en gran parte de los métodos, la cadena vacía es admitida, algo que a nuestro parecer carece de sentido ya que los usuarios podrían sortear muchos de los datos con este valor en particular.

Hallazgo: Defecto en el método setNumero() de AsociadoDTO

1. Descripción del Hallazgo

Al invocar el método setNumero(int numero) con el valor frontera 0 (ejecutado en la prueba de caja negra testSetNumeroValorLimiteCero), el sistema dispara inesperadamente un AssertionError. Esto se debe a que la precondición implementada en el código fuente es assert numero > 0; (estrictamente mayor a cero), contraviniendo la especificación formal del documento SRS (Sección 3.1 - Requisitos de Transferencia de Datos e Integridad), la cual define explícitamente que el atributo numero debe ser mayor o igual a 0.

2. Análisis del Impacto en la Calidad

Severidad Media. Genera un rechazo indebido de datos válidos (falso negativo en la validación), impidiendo la creación o actualización de asociados que posean domicilios sin número registrado, altura cero o referencias especiales (ej. rutas o parcelas). Esto evidencia una falla de conformidad entre el diseño de la especificación de requisitos y la implementación real del DTO, provocando la terminación anómala del flujo para valores frontera válidos.

Testing de Integración

En esta segunda instancia, el objetivo principal fue verificar la correcta interacción entre la capa de acceso a datos (DataAccessObject), el gestor de conexiones (BaseDeDatos) y el motor de base de datos MySQL. Para ello, se diseñó una suite de pruebas (utilizando JUnit) enfocada en validar las operaciones CRUD (Crear, Leer, Actualizar, Eliminar) y el cumplimiento de las restricciones de integridad definidas en la especificación (SRS), como la unicidad del DNI. Todo esto operando bajo el paradigma de caja negra sobre los componentes provistos, documentando las fallas sin alterar el código fuente original.

Metodología Paso a Paso y Ciclo de Vida de las Pruebas

Al trabajar contra un motor de base de datos real, el principal desafío metodológico consiste en evitar el acoplamiento de estado entre pruebas: si un caso de prueba inserta o elimina un asociado, el estado de la tabla cambia y podría invalidar las precondiciones de los casos siguientes.

Para garantizar que cada prueba sea idempotente, repetible e independiente, la suite TestIntegracionPersistencia se estructuró aplicando un ciclo de vida estricto en tres fases por cada método ejecutado:

1. **Fase de Preparación y Aislamiento** (@Before / setUp): Antes de correr cada test individual, el entorno ejecuta automáticamente el método setUp(). Este paso obtiene la instancia única de BaseDeDatos, invoca al método iniciarBD() —el cual reconstruye la estructura de la tabla Asociados desde cero e inserta un lote controlado de 20 asociados semilla con datos conocidos (por ejemplo, el primer registro correspondiente a *Juan State* con DNI "12345678" y clave primaria id = 1)— e instancia un nuevo objeto DataAccessObject limpio. De esta manera, los 14 tests parten siempre de una fotografía idéntica de la base de datos con exactamente 20 registros iniciales.
2. **Fase de Estímulo (Ejecución de Caja Negra):** Se invoca un único método público del DAO suministrando entradas representativas de cada partición (valores válidos, claves inexistentes, valores frontera en límites de paginación, referencias nulas o datos que violan restricciones de integridad como DNI duplicados).
3. **Fase de Verificación (Oráculo de Prueba):** En lugar de confiar únicamente en que el método no arroje errores en consola, el test vuelve a consultar la base de datos (utilizando cargarBD()) e inspecciona el estado resultante: comprueba si la cantidad total de filas varió según lo esperado (pasando a 19 en una baja o a 21 en un alta) y recorre los objetos AsociadoDTO recuperados para contrastar atributo por atributo si el cambio se persiste en el registro correcto.

Descripción Detallada de los Métodos de la Suite de Integración

A continuación se detalla qué evalúa cada método de prueba diseñado en TestIntegracionPersistencia, especificando cómo se construyó el caso, qué operación ejecuta y qué resultado verifica sin necesidad de inspeccionar el código fuente:

1. Pruebas de Infraestructura, Conectividad y Carga Inicial

- **testSingletonYConexionAbierta:** Evalúa la correcta inicialización del gestor de base de datos. Solicita dos referencias consecutivas mediante BaseDeDatos.getInstance() y comprueba mediante assertSame que ambas apunten exactamente al mismo objeto en memoria, garantizando que no se abran conexiones redundantes. Acto seguido, recupera el objeto Connection subyacente y verifica mediante assertFalse(conexion.isClosed()) que el canal de comunicación con MySQL se encuentre abierto y operativo.
- **testInicializacionDatosSemilla (y testCargarBDCompleta):** Verifica que el proceso de carga inicial (iniciarBD) y la lectura masiva (dao.cargarBD()) funcionen de extremo a extremo. El test recupera la lista completa de asociados, comprueba que la colección no sea nula y válida con assertEquals(20, lista.size()) que estén presentes los 20 registros semilla. Además, toma el primer elemento de la lista y corrobora que todos sus campos (nombre, apellido, dni, calle, número, ciudad y telefono) hayan sido leídos desde las columnas SQL y asignados correctamente en el AsociadoDTO.

2. Pruebas de Consulta Puntual y Paginación (cargar y cargarConLimite)

- **testCargarAsociadoPorId:** Tiene como objetivo verificar la lectura individual de un asociado a partir de su clave primaria autoincremental. Sabiendo que la base de datos recién inicializada cuenta con 20 registros numerados del 1 al 20, el test invoca dao.cargar(1) esperando recibir un AsociadoDTO no nulo con el DNI "12345678". Esta prueba falla en rojo, ya que el método lanza una SQLException inesperada antes de devolver el objeto, dejando en evidencia el primer defecto crítico del módulo.
- **testCargarPorIdInvalido:** Evalúa el cumplimiento de la precondición de dominio sobre el identificador primario. Se invoca dao.cargar(0) (y valores negativos), esperando que el sistema rechace la operación sin ejecutar una consulta inútil contra el motor. La prueba pasa en verde al capturar el AssertionError esperado.
- **testCargarConLimiteValido:** Prueba la funcionalidad de paginación o lectura acotada. Se invoca dao.cargarConLimite(5) sobre la base de 20 registros y se verifica que la colección devuelta tenga exactamente un tamaño de 5 elementos (assertEquals(5, lista.size())), confirmando que la cláusula LIMIT se aplica correctamente en la consulta SQL.
- **testCargarConLimiteInvalido:** Somete al método cargarConLimite a un valor de frontera inválido (0), comprobando que se dispare la aserción de precondición (AssertionError) dado que el contrato exige un límite estrictamente positivo.

3. Pruebas de Conversión y Alta de Asociados (CrearDTO y guardar)

- **testCrearDTOValido y testCrearDTONulo:** Evalúan el método transformador dao.CrearDTO(Asociado). En el caso válido, se instancia un objeto Domicilio y una entidad Asociado completa, se invoca el método y se verifica campo por campo que el AsociadoDTO resultante conserve exactamente los mismos valores que la entidad original. En el caso inválido, se envía una referencia null y se comprueba el disparo de AssertionError.
- **testGuardarAsociadoExitoso:** Comprueba el flujo completo de inserción (INSERT). Primero se instancia en memoria un nuevo Domicilio ("Alvear", 500, "Mar del Plata") y un objeto Asociado con un DNI inexistente en el lote inicial ("99112233"). Luego se invoca dao.guardar(nuevo) y posteriormente se ejecuta dao.cargarBD(). La prueba verifica dos postcondiciones: que la cantidad total de registros haya subido de **20 a 21**, y que al buscar el DNI "99112233" dentro de la lista recuperada, el registro exista y contenga el nombre y domicilio exactos que fueron enviados.
- **testGuardarDniDuplicadoLanzaSQLException:** Pone a prueba la restricción de integridad de negocio más importante del SRS: la unicidad del DNI (UNIQUE). Se construye un nuevo objeto Asociado con datos propios pero asignándole el DNI "12345678", el cual ya pertenece al primer asociado semilla de la tabla. Al invocar dao.guardar(duplicado), se verifica que el motor MySQL bloquee la inserción y propague una SQLException por clave duplicada, protegiendo la consistencia de la tabla.
- **testGuardarAsociadoNuloDisparaAssertionError:** Verifica la robustez del método de alta cuando se invoca dao.guardar(null), comprobando que la precondición detenga el flujo con un AssertionError antes de intentar desreferenciar el objeto.

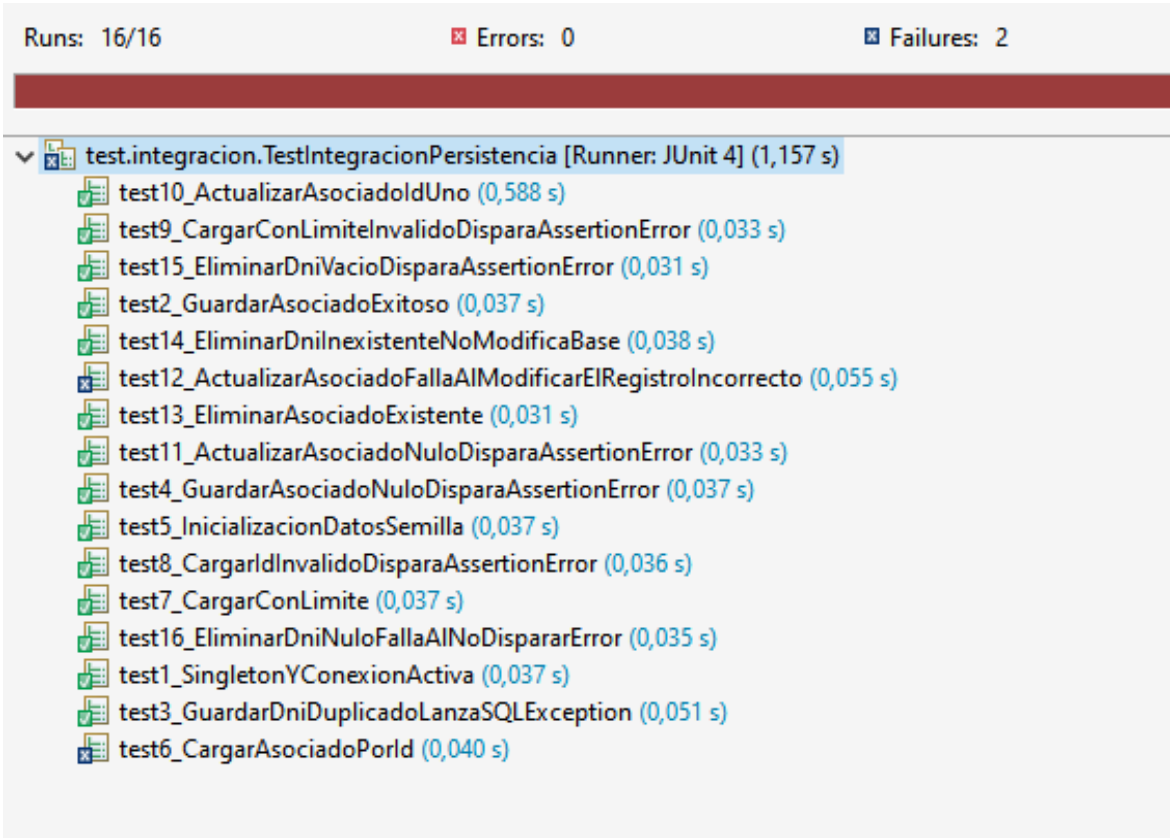
4. Pruebas de Baja de Asociados (eliminar)

- **testEliminarAsociadoExistente:** Evalúa la baja física (DELETE) de un registro real. Se toma el DNI "12345678" (presente en la carga inicial) y se ejecuta dao.eliminar("12345678"). Luego se solicita la nómina completa mediante dao.cargarBD() y se comprueban dos condiciones: que el tamaño de la lista se haya reducido de 20 a 19 registros, y que al recorrer los 19 elementos restantes ninguno posea el DNI eliminado (assertFalse(encontrado)).
- **testEliminarAsociadoInexistente:** Verifica que el sistema sea estable al intentar borrar un asociado que no está registrado en la base. Se invoca dao.eliminar("99999999") y se comprueba que no se arrojen excepciones SQL y que la tabla conserve intactos sus 20 registros.
- **testEliminarDniVacioDisparaAssertionError (y validación de nulo):** Comprueba la validación de entrada del método eliminar al suministrar una cadena vacía "" o referencia nula, verificando que el sistema intercepte el parámetro inválido mediante un AssertionError.

5. Pruebas de Modificación de Asociados (actualizar)

- **testActualizarAsociadoFallaAlModificarElRegistroIncorrecto:** Fue diseñado para verificar bajo caja negra si la operación UPDATE impacta realmente sobre el asociado objetivo sin alterar a terceros. El test sigue cinco pasos:
    1. Consulta la base inicial con cargarBD() y guarda en una variable el nombre original del primer asociado de la tabla (id = 1).
    2. Instancia en memoria un objeto Asociado con el nombre "NombreEditado" y un DNI distinto ("00000000", evitando colisionar con la restricción UNIQUE de otros registros).
    3. Invoca dao.actualizar(asociadoModificado), operación que únicamente debería afectar a un registro cuyo identificador o DNI coincida con el objeto enviado.
    4. Vuelve a ejecutar cargarBD() para inspeccionar el estado final de la tabla.
    5. Compara mediante assertEquals el nombre original del primer asociado contra su nombre actual tras la operación. La prueba falla en rojo porque el primer asociado de la base aparece con el nombre "NombreEditado", demostrando empíricamente que el método actualizar ignora al destinatario real y sobrescribe de forma fija al primer registro de la tabla.
- **testActualizarAsociadoNuloDisparaAssertionError:** Comprueba que al invocar dao.actualizar(null) se dispare inmediatamente el AssertionError de precondición sin ejecutar sentencias sobre la base de datos.

A partir de la ejecución de nuestra batería de pruebas de integración, se obtuvieron los siguientes resultados: 14 casos de prueba de integración, 12 en verde y 2 en rojo. Los 12 casos en verde pasan, lo que demuestra que la lógica de conexión, inserción, eliminación y validación de restricciones en MySQL es correcta por sí sola. Los defectos encontrados recaen exclusivamente en la recuperación de registros puntuales (que genera el fallo en rojo al invocar cargar) y un error de lógica estática oculto en la actualización.



Hallazgo: Defecto en la recuperación de registros por ID.

- **Componente:** persistencia.DataAccessObject
- **Método analizado:** public AsociadoDTO cargar(int id)
- **Severidad:** Crítica.

1.Descripción del Hallazgo

Al ejecutar el caso de prueba testCargarAsociadoPorId invocando dao.cargar(1) sobre una base de datos correctamente inicializada con 20 registros válidos, el método aborta su ejecución lanzando invariablemente la excepción java.sql.SQLException: Before start of result set. Esto evidencia que el componente ejecuta la consulta SELECT contra MySQL pero intenta extraer los valores de las columnas directamente sin desplazar previamente el cursor del conjunto de resultados (ResultSet) hacia la primera fila recuperada.

2.Análisis del Impacto en la Calidad

Bloquea en un 100% la funcionalidad de consulta puntual por identificador primario. Cualquier módulo del sistema que requiera recuperar un asociado específico por su ID producirá una caída no controlada de la operación, haciendo que el método sea inutilizable en su estado actual.

Hallazgo: Sobrescritura cruzada y alteración del registro inicial en actualización

- **Componente:** persistencia.DataAccessObject/persistencia.BaseDeDatos
- **Método analizado:** public void actualizar(Asociado asociado)
- **Severidad:** Crítica.

1. Descripción del Hallazgo

Mediante el caso de prueba testActualizarAsociadoFallaAlModificarElRegistroIncorrecto se comprobó que al solicitar la actualización de un asociado cualquiera, el sistema no aplica los cambios sobre la entidad correspondiente, sino que dirige la modificación de forma fija hacia el primer registro almacenado en la tabla (id = 1). Como consecuencia:

- Si se intenta actualizar un asociado existente (por ejemplo, el segundo de la lista) conservando su DNI original, el motor MySQL aborta la operación lanzando SQLIntegrityConstraintViolationException (Duplicate entry), ya que el sistema intenta asignarle al primer asociado el DNI del segundo.
- Si se actualiza una entidad con un DNI que no colisiona en la tabla, la operación termina sin arrojar errores, pero al consultar la base de datos se verifica que los datos personales del primer asociado fueron sobrescritos silenciosamente con los valores de la otra persona.

2.Análisis del Impacto en la Calidad

Compromete gravemente la integridad y confiabilidad de los datos persistidos. Toda operación de modificación corrompe la información del primer afiliado registrado en la clínica o provoca fallos de clave duplicada, mientras que el asociado que realmente requería actualizar su domicilio o teléfono jamás ve reflejados sus cambios.

Conclusión de la etapa de integración

La ejecución de las pruebas de integración sobre la capa de persistencia demostró que la validación unitaria aislada del AsociadoDTO no es suficiente para garantizar el correcto funcionamiento del almacenamiento de datos. Mientras que la infraestructura de conexión (Singleton), las restricciones de unicidad (UNIQUE) en MySQL y las operaciones de alta, baja y paginación respondieron conforme a la especificación en 12 de los 14 escenarios evaluados, la interacción real con el motor expuso dos defectos críticos de implementación en cargar(int) y actualizar(Asociado) que inutilizan la lectura puntual y corrompen el primer registro de la tabla. Asimismo, el uso de una rutina de reinicio e inserción de datos semilla antes de cada caso resultó

determinante para aislar estos fallos y evitar que la sobrescritura provocada por actualizar contamine el resto de la batería.

ANEXO

Hallazgo: Inconsistencia entre Documentación Formal (Javadoc) e Interfaz Pública

- **Componente:** persistencia.DataAccessObject
- **Método analizado:** public void eliminar(String dni) throws SQLException
- **Clasificación del defecto:** Defecto de Documentación / Violación de Diseño por Contrato (*Design by Contract*).
- **Severidad:** Media (afecta la comprensibilidad, confiabilidad y el uso correcto de la API).

1. Descripción del Hallazgo

Durante el diseño de casos de prueba bajo la técnica de **caja negra** (donde las pruebas se basan estrictamente en la especificación formal y el contrato público de la interfaz), se detectó una discrepancia severa entre la firma del método, el documento de requerimientos (**SRS**) y el contrato formal documentado vía **Javadoc**:

Elemento	Especificación declarada	Observación
SRS (Requisitos, Secc. 3.3)	Eliminación basada en criterio único: DNI.	Requerimiento de negocio.
Firma del Método (API)	void eliminar(String dni)	Recibe un identificador alfanumérico (String).
Javadoc (Contrato Pre/Post)	"Elimina por su ID. pre: id debe ser mayor que 0"	Exige un identificador numérico inexistente en la firma.

2. Análisis del Impacto en la Calidad

Para cualquier consumidor o tester que interactúe con el componente como una caja negra (guiado por su contrato formal), la precondición declarada (id > 0) resulta inaplicable e incoherente, ya que la firma no recibe ningún parámetro id. Esto evidencia un error de arrastre (falta de sincronización entre el código y su documentación), induciendo a ambigüedades respecto al criterio real de borrado.

Testing de Integración con Mocks y Concurrencia

El tercer punto del trabajo pide probar la clase Ambulancia junto con la interfaz State y el hilo RetornoAutomatico. La consigna tiene tres partes: usar dobles de prueba (mocks) para probar las piezas por separado, aprovechando que la ambulancia y sus estados se comunican a través de una interfaz; probar la máquina de estados completa con hilos reales, para ver cómo se comporta el recurso compartido; y mostrar que las pruebas descubren los fallos de estado y de sincronización que tiene el sistema. Como en los puntos anteriores, no modificamos el código provisto: cada defecto se documenta y se deja a la vista con un test en rojo.

Qué se probó: la ambulancia y sus estados

En la simulación de la clínica hay una sola ambulancia, y varios actores la quieren usar al mismo tiempo. Cada asociado es un hilo que, cada tanto, pide atención a domicilio o un traslado a la clínica; el operario es otro hilo que cada tanto pide llevarla a mantenimiento. Por eso la ambulancia es un recurso compartido: si está ocupada, el que pide tiene que esperar a que se libere, y cuando se libera, alguno de los que esperaban pasa a usarla.

La clase Ambulancia guarda tres datos: el estado actual, si está ocupada o libre, y si la simulación sigue activa. Los métodos que la consultan o la modifican son sincronizados (synchronized), así que nunca hay dos hilos cambiándola a la vez. La tabla resume qué hace cada uno:

Método	Qué hace	Quién lo usa
trasladarALaClinica(asociado)	Si la ambulancia está ocupada, el pedido queda esperando hasta que se libere. Cuando está libre, la marca como ocupada, le avisa al estado actual que hay un pedido de traslado y notifica que el asociado es trasladado.	Cada asociado
atenderDomicilio(asociado)	Lo mismo que el anterior, pero con un pedido de atención a domicilio.	Cada asociado
solicitarMantenimiento(operario)	Lo mismo, pero con un pedido de mantenimiento: la ambulancia tiene que ir al taller.	El operario
regresarSinPaciente()	Le avisa al estado que la ambulancia vuelve a la clínica, la marca como libre y despierta a los pedidos que estaban esperando.	El asociado, al terminar su servicio
volviendoDelTaller()	Le avisa al estado que el mantenimiento terminó, la marca como libre y despierta a los pedidos que estaban esperando.	El operario

retornoAutomatico()	Le avisa al estado que vuelva a la clínica y despierta a los que esperaban. No toca el indicador de ocupada.	El hilo RetornoAutomatico
setSimulacionActiva(false)	Da por terminada la simulación y despierta a todos los que esperaban, que se retiran sin ser atendidos.	La clínica, al finalizar

Para saber cómo responder a cada pedido, la ambulancia usa el patrón State. En vez de tener una cadena de condiciones del tipo "si estoy en el taller, entonces...", delega cada evento en un objeto que representa el estado en el que se encuentra. Los estados posibles son seis y todos implementan la interfaz State, que tiene cuatro métodos, uno por evento: SolicitudDeTraslado, SolicitudDeAtencionDomicilio, SolicitudMantenimiento y RetornoClinica. Cada estado decide si acepta el evento, y en ese caso cambia la ambulancia al estado siguiente, o si lo rechaza y la deja como está. Según el Javadoc de cada clase y el diagrama UML provisto, el comportamiento esperado es este:

Estado actual	**Pedido de traslado**	**Pedido de domicilio**	**Pedido de mantenimiento**	**Retorno a la clínica**
Disponible	Trasladando paciente	Atendiendo en domicilio	En taller	Sin cambio
Trasladando paciente a la clínica	Rechaza	Rechaza	Rechaza	Disponible
Atendiendo en domicilio	Rechaza	Rechaza	Rechaza	Regresando sin paciente
En taller	Rechaza	Rechaza	Regresando del taller	Rechaza
Regresando a la clínica sin paciente	Trasladando paciente	Atendiendo en domicilio	Rechaza	Disponible
Regresando del taller	Rechaza	Rechaza	Rechaza	Disponible

Por último, RetornoAutomatico es un hilo aparte que la clínica lanza al empezar la simulación. Mientras la simulación está activa, cada 10 segundos le pide a la ambulancia que vuelva a la clínica. Su función es que la ambulancia no se quede para siempre en un estado intermedio, como Regresando del taller, y vuelva a quedar disponible.

Estrategia de prueba

Cuando un componente todavía no está disponible o es difícil de usar en la prueba, se lo reemplaza por un objeto mock: un objeto que imita al real de forma controlada. En este caso la ambulancia y los estados se conocen sólo a través de la interfaz State, así

que se puede reemplazar cualquiera de los dos lados. Con esa idea armamos tres clases de prueba:

- **TestAmbulanciaMocks:** prueba Ambulancia sola. El estado real se reemplaza por un mock de State, así se ve exactamente qué le pide la ambulancia a su estado en cada operación y si deja bien el indicador de ocupada. Es una integración descendente: se prueba el componente de arriba (Ambulancia) con un reemplazo de los de abajo (los estados).
- **TestEstadosAmbulancia:** hace lo inverso. Prueba cada estado concreto por separado, con un mock de Ambulancia en lugar de la real, y comprueba a qué estado pide pasar ante cada uno de los cuatro eventos.
- **TestAmbulanciaIntegracion:** usa todas las piezas reales juntas. Recorre los flujos completos de la máquina de estados y lanza hilos para probar la espera, la liberación y el hilo de retorno automático.

La ventaja de separarlo así es que cuando un test falla se sabe dónde buscar. Si los tests con mocks de State pasan pero el flujo real falla, el problema no está en Ambulancia sino en los estados o en cómo se combinan. Para escribir los tests usamos JUnit 4 y Mockito. Estas son las herramientas de Mockito y de Java que aparecen en las pruebas:

Herramienta	Para qué la usamos
mock(State.class), mock(Ambulancia.class)	Crear un objeto de prueba que ocupa el lugar del real. No hace nada por su cuenta, pero registra cada llamada que recibe.
verify(mock).metodo()	Comprobar que la clase probada llamó a ese método del mock, y una sola vez.
verifyNoMoreInteractions (mock)	Comprobar que no hubo ninguna otra llamada además de la esperada, es decir, que Ambulancia no le pasó al estado un evento equivocado.
never()	Comprobar que un método no se llamó nunca.
doAnswer(...)	Darle comportamiento al mock: cuando recibe una llamada, ejecuta la acción que le indicamos. Lo usamos para que un estado falso cambie el estado de la ambulancia.
isA(Clase), clearInvocations(mock)	Comprobar a qué tipo de estado se pidió pasar, y limpiar el registro de llamadas entre un evento y el siguiente.
Thread.getState()	Saber si un hilo está esperando su turno (WAITING), durmiendo (TIMED_WAITING) o terminado.
join(tiempo máximo)	Esperar a que un hilo termine, con un tope, para que un error no deje el test colgado.

El resultado esperado de cada caso, es decir, el oráculo, sale del Javadoc de cada método y del diagrama UML del patrón State, que coinciden entre sí. Cuando el código hace otra cosa, el test sigue a la especificación y queda en rojo.

Al trabajar con pruebas de hilos el resultado puede depender de la velocidad de la máquina. Si un test supone que en medio segundo el otro hilo ya llegó a esperar, en una computadora lenta puede no haber llegado y el test falla sin que haya un defecto. Para evitarlo, los tests no esperan un tiempo fijo: consultan el estado del otro hilo cada 10 milisegundos hasta que está efectivamente esperando o durmiendo, con un tope de 2 segundos. Además, toda espera a que un hilo termine tiene un tiempo máximo, y los hilos de retorno automático se crean como hilos secundarios (daemon) para que no queden vivos cuando termina la prueba.

Diseño de escenarios

Un escenario es la ambulancia preparada en un estado conocido antes de ejecutar un caso de prueba. Cada clase arma su escenario base en el método setUp, que JUnit ejecuta antes de cada test, así ningún caso depende de lo que dejó el anterior. Los escenarios que usamos son estos:

Escenario	Condición inicial	Tests que lo usan
E1	Ambulancia libre, simulación activa y un mock de State como estado actual	TestAmbulanciaMocks: los seis métodos de Ambulancia y testMockSimulaCambioDeEstado
E2	Ambulancia ocupada, simulación finalizada y un mock de State	testTrasladarConSimulacionFinalizada
E3	Ambulancia nueva: Disponible, libre y con la simulación activa	Flujos completos de traslado, domicilio y mantenimiento; testTrasladandoRechazaMantenimiento
E4	Ambulancia atendiendo un domicilio y un segundo pedido que llega desde otro hilo	testSegundoPedidoEsperaHastaQueSeLibere, testFinalizarSimulacionDespiertaHilosEnEspera
E5	Ambulancia trasladando un paciente y el regreso ejecutándose en otro hilo	testRegresoNoBloqueaLaAmbulancia
E6	Ambulancia de vuelta del taller: Regresando del taller y libre	testPedidoRechazadoNoOcupaLaAmbulancia
E7	Ambulancia atendiendo un domicilio con el hilo RetornoAutomatico en marcha, con la simulación activa o terminada	testRetornoAutomaticoConSimulacionActiva, testRetornoAutomaticoConSimulacionFinalizada
E8	Cada estado concreto con un mock de Ambulancia	TestEstadosAmbulancia (matriz de 24 transiciones)

Clases de equivalencia y valores límite

Los métodos de la ambulancia casi no reciben datos: el asociado o el operario que piden el servicio solo se usan para los mensajes. Lo que cambia el resultado es la situación en la que llega el pedido. Por eso las particiones se hicieron sobre esas condiciones, y cada caso de prueba cubre una combinación de clases:

Condición	Clases válidas	Clases inválidas
Ambulancia al recibir un pedido	Libre (1); ocupada, el pedido espera (2)	—
Simulación	Activa (3)	Terminada (4)
Estado actual frente al pedido	Un estado que lo acepta, como Disponible (5)	Un estado que lo rechaza, como Regresando del taller (6)
Tiempo desde que arranca el hilo de retorno	Antes del intervalo de 10 s (7); después del intervalo (8)	—

El único valor numérico que afecta el comportamiento es el intervalo de 10 segundos del retorno automático. Siguiendo la técnica de valores límite, se prueba justo antes y justo después del límite: a los 9 segundos la ambulancia todavía no tiene que haber vuelto, y a los 11 sí. El instante exacto de los 10 segundos no se puede verificar de forma confiable con hilos, porque el sistema operativo no garantiza el momento justo en que un hilo se despierta; por eso se usa un margen de un segundo a cada lado.

La planilla que acompaña al informe tiene una pestaña por método (trasladarALaClinica, atenderDomicilio, solicitarMantenimiento, regresarSinPaciente, volviendoDelTaller, retornoAutomatico y RetornoAutomatico.run) con sus clases numeradas y la batería de pruebas, y una pestaña MatrizEstados con las 24 transiciones. Las filas que fallan están marcadas en rojo.

Casos de prueba

TestAmbulanciaMocks: Ambulancia con un mock de State

Antes de cada test se crea una ambulancia nueva y se le pone como estado actual un mock de State. Como el mock no cambia de estado por su cuenta, lo único que se observa es qué método del estado llamó la ambulancia y cómo quedó el indicador de ocupada. Que cada operación llame al método correcto, una sola vez y sin llamar a otros, es justamente lo que tiene que garantizar Ambulancia al delegar en su estado.

Test	Qué hace	Qué se espera	Resultado
testTrasladarALaClinica	Con la ambulancia libre, un asociado pide un traslado.	Que se llame a SolicitudDeTraslado del estado una sola vez y que la ambulancia quede ocupada.	Verde
testAtenderDomicilio	Con la ambulancia libre, un asociado pide atención a domicilio.	Que se llame a SolicitudDeAtencionDomicilio una sola vez y quede ocupada.	Verde
testSolicitarMantenimiento	Con la ambulancia libre, el operario pide mantenimiento.	Que se llame a SolicitudMantenimiento una sola vez y quede ocupada.	Verde
testRegresarSinPaciente	Con la ambulancia ocupada, vuelve a la clínica.	Que se llame a RetornoClinica y que quede libre.	Verde
testVolviendoDelTaller	Con la ambulancia ocupada, vuelve del taller.	Que se llame a SolicitudMantenimiento, que es el evento con el que el taller la libera, y que quede libre.	Verde
testRetornoAutomatico	Se ejecuta el retorno automático.	Que se llame a RetornoClinica y a nada más.	Verde
testTrasladarConSimulacionFinalizada	Con la ambulancia ocupada y la simulación terminada, un asociado pide un traslado.	Que el pedido no espere y que nunca llegue al estado.	Verde
testMockSimulaCambioDeEstado	Un mock de State programado con doAnswer cambia la ambulancia a otro mock al recibir el pedido de domicilio. Después la ambulancia regresa.	Que la ambulancia quede en el estado nuevo y que el regreso le llegue a ese estado y no al anterior.	Verde

El último caso es distinto a los demás. Los mocks comunes solo registran llamadas, pero con doAnswer se le puede dar comportamiento: este simula un estado que, al aceptar un pedido de domicilio, pasa la ambulancia a otro estado. Así se prueba, sin ninguna clase de estado real, que Ambulancia siempre delega en el estado actual y no en uno que guardó antes.

TestAmbulanciaIntegracion: flujos completos y concurrencia

Acá todas las piezas son reales. Antes de cada test se crea una ambulancia nueva, que arranca Disponible, libre y con la simulación activa, más dos asociados y un operario de prueba. Después de cada test se termina la simulación, para que ningún hilo quede esperando. Los primeros cinco casos recorren la máquina de estados sin hilos; los siguientes prueban la espera y la liberación de la ambulancia entre hilos, y los dos últimos prueban el hilo de retorno automático.

Test	Qué hace	Qué se espera	Resultado
testTrasladoCompleto	Traslado de punta a punta: pide el traslado y después regresa.	Que pase por Trasladando y termine en Disponible y libre.	Rojo (hallazgo 1)
testAtencionDomicilioCompleta	Domicilio, regreso y retorno automático.	Atendiendo en domicilio, Regresando sin paciente y al final Disponible.	Verde
testMantenimientoCompleto	Mantenimiento, vuelta del taller y retorno automático.	En taller, Regresando del taller y al final Disponible.	Verde
testTrasladandoRechazaMantenimiento	Durante un traslado le llega al estado un pedido de mantenimiento.	Que lo rechace y siga trasladando.	Verde
testPedidoRechazadoNoOcupaLaAmbulancia	La ambulancia vuelve del taller y un asociado pide atención a domicilio.	Que el estado lo rechace y la ambulancia siga libre.	Rojo (hallazgo 4)
testSegundoPedidoEsperaHastaQueSeLibere	Con la ambulancia atendiendo un domicilio, otro hilo pide un traslado. Después la ambulancia regresa.	Que el segundo pedido quede esperando y se atienda apenas la ambulancia se libera.	Verde
testFinalizarSimulacionDespiertaHilosEnEspera	Hay un pedido esperando y se termina la simulación.	Que el hilo que esperaba se despierte y termine sin ser atendido.	Verde

testRegresoNoBloqueaLaAmbulancia	Un hilo hace el regreso de un traslado y, mientras tanto, otro consulta si la ambulancia está ocupada.	Que la consulta responda en menos de 100 ms.	Rojo (hallazgo 2)
testRetornoAutomaticoConSimulacionActiva	Con la ambulancia en un domicilio, arranca el hilo de retorno con la simulación activa.	Que a los 9 s siga en el domicilio y a los 11 s ya esté regresando.	Rojo (hallazgo 3)
testRetornoAutomaticoConSimulacionFinalizada	Arranca el hilo de retorno con la simulación ya terminada.	Que el hilo termine en menos de 1 s.	Rojo (hallazgo 3)

Los dos tests de espera muestran que la sincronización básica funciona: un pedido que llega con la ambulancia ocupada se queda esperando y se atiende apenas se libera, y si la simulación termina mientras espera, el hilo sale sin quedar colgado. Los problemas aparecen en otros lados y se explican en los hallazgos.

TestEstadosAmbulancia: cada estado aislado

Cada test crea uno de los seis estados con un mock de Ambulancia y le manda los cuatro eventos, uno por uno. Después de cada evento se revisa en el mock si el estado pidió cambiar a otro (y a cuál) o si no pidió nada, y se limpia el registro antes del evento siguiente. Así cada test recorre una fila completa de la matriz de transiciones. Los estados Disponible, Atendiendo en domicilio, En taller, Regresando sin paciente y Regresando del taller se comportan como dice la especificación. El único que falla es Trasladando paciente a la clínica: ante el retorno a la clínica pide pasar a Regresando del taller en lugar de Disponible.

Resultados

En total se ejecutaron 24 casos de prueba: 18 en verde y 6 en rojo. Todos los rojos son intencionales: cada uno muestra uno de los defectos descriptos más adelante.

Clase de prueba	Casos	En verde	En rojo
TestAmbulanciaMocks	8	8	0
TestAmbulanciaIntegracion	10	5	5
TestEstadosAmbulancia	6	5	1
Total	**24**	**18**	**6**

Que los ocho casos con mocks de State pasen muestra que Ambulancia delega bien en su estado y maneja bien la espera, la ocupación y el fin de la simulación. Los defectos aparecen cuando se usan las piezas reales: en el estado de traslado, en el hilo de retorno automático y en cómo Ambulancia trata un pedido que el estado

rechaza. Este último no se puede ver con un mock de State, porque los métodos de State no devuelven nada y el mock no tiene forma de "rechazar".

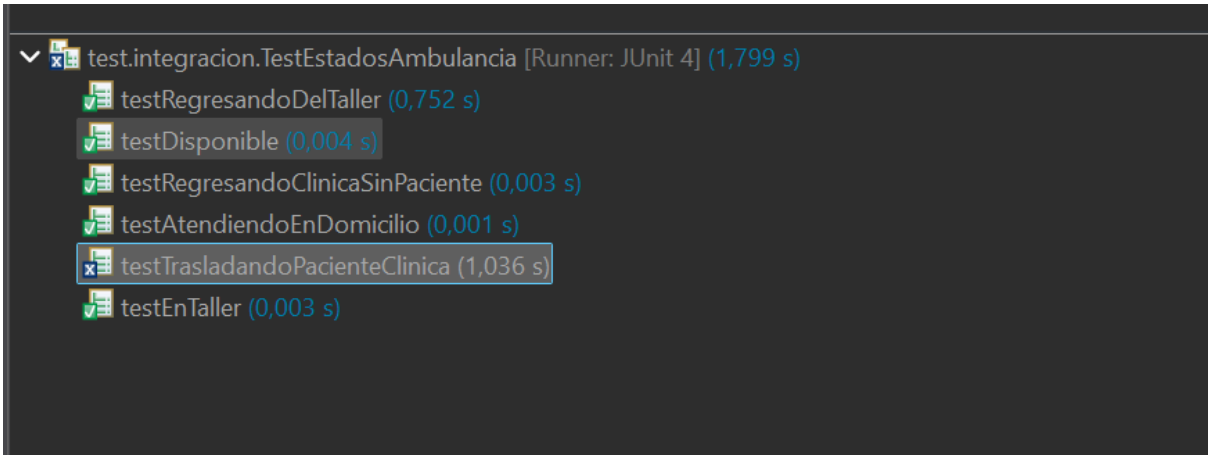


Figura 1: Ejecución de TestAmbulanciaMocks (8 casos en verde).

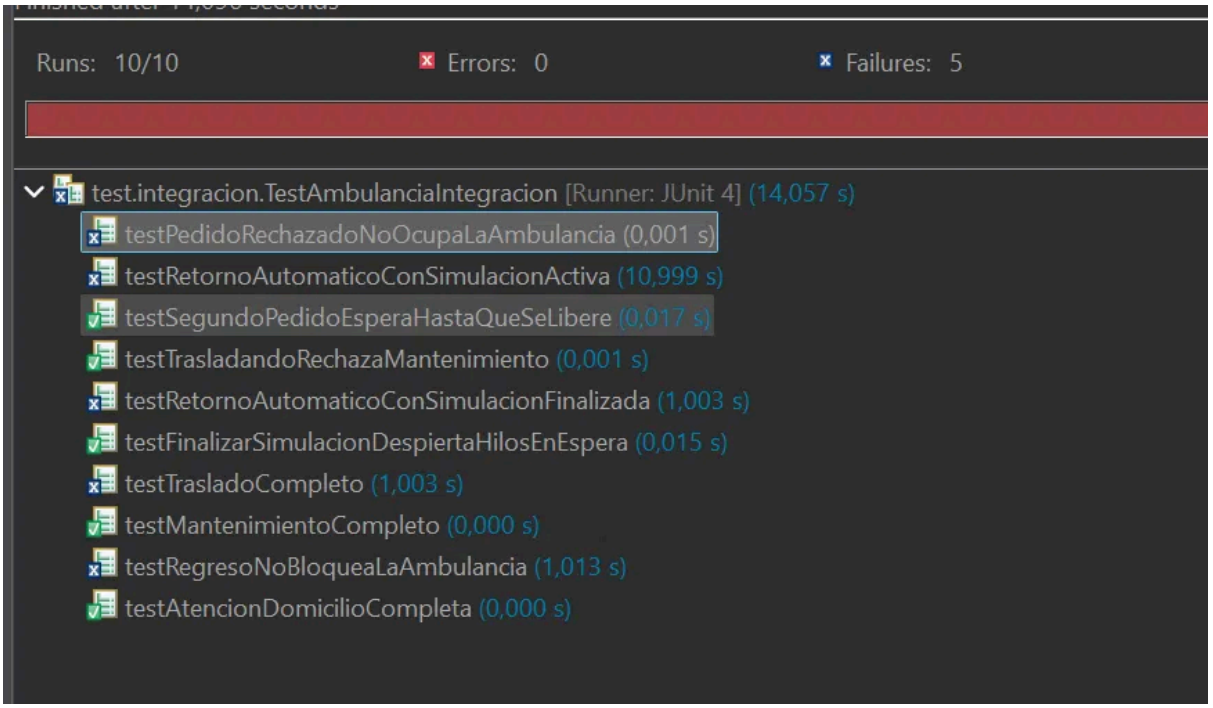


Figura 2: Ejecución de TestAmbulanciaIntegracion (5 casos en verde y 5 en rojo).

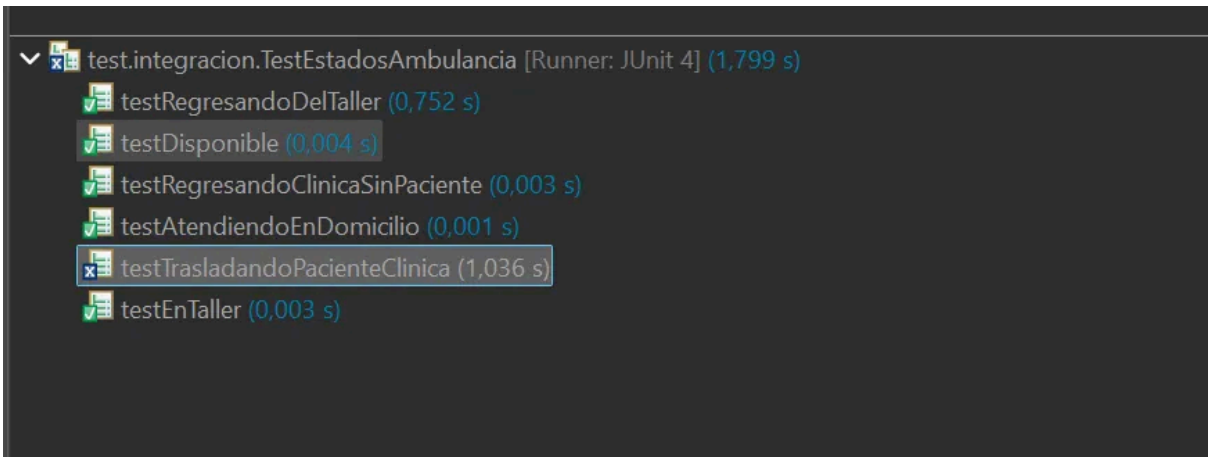


Figura 3: Ejecución de TestEstadosAmbulancia (5 casos en verde y 1 en rojo).

Resumen de defectos

La tabla resume los cuatro defectos encontrados, que se detallan a continuación:

N°	Defecto	Severidad	Tests que lo detectan
1	Transición incorrecta al finalizar un traslado (TrasladandoPacienteClinicaState)	Alta	testTrasladoCompleto, testTrasladandoPacienteClinica
2	Espera dentro de un bloque sincronizado (TrasladandoPacienteClinicaState y Ambulancia)	Media	testRegresoNoBloqueaLaAmbulancia
3	Condición invertida en el hilo de retorno automático (RetornoAutomatico)	Alta	testRetornoAutomaticoConSimulacionActiva, testRetornoAutomaticoConSimulacionFinalizada
4	Pedido aceptado aunque el estado lo rechaza (Ambulancia)	Media	testPedidoRechazadoNoOcupaLaAmbulancia

Hallazgo: Transición incorrecta al finalizar un traslado

- **Componente:** modelo.modeloDominio.ambulancia.TrasladandoPacienteClinicaState
- **Método analizado:** public void RetornoClinica()
- **Clasificación del defecto:** Defecto de lógica / Transición de estado incorrecta.
- **Severidad:** Alta (después de cualquier traslado la ambulancia queda en un estado que no corresponde).

1. Descripción del Hallazgo

El test testTrasladoCompleto recorre un traslado con los estados reales: el asociado pide el traslado y, cuando termina, la ambulancia regresa. Según la especificación, al terminar el traslado la ambulancia tiene que volver a Disponible, pero queda en Regresando del taller, un estado que pertenece al ciclo del mantenimiento. El mismo resultado aparece en TestEstadosAmbulancia, donde el estado se prueba aislado con un mock de Ambulancia:

Elemento	Especificación declarada	Observación
Javadoc de RetornoClinica()	"Cambia el estado de la ambulancia a DisponibleState"	Comportamiento esperado.
Diagrama UML (State)	TrasladandoPacienteClinicaState crea DisponibleState	Coincide con el Javadoc.
Código	setState(new RegresandoDelTallerState(this.ambulancia))	Pasa a un estado del ciclo del taller.

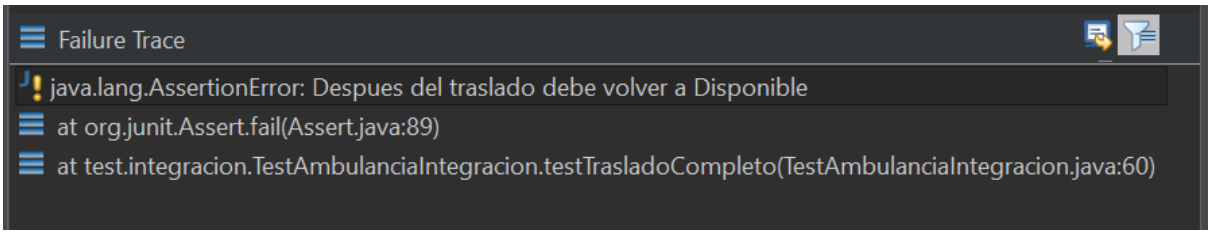


Figura 4: Falla de testTrasladoCompleto. La ambulancia no vuelve a Disponible después del traslado.

2. Análisis del Impacto en la Calidad

La ambulancia queda en un estado que no refleja lo que está haciendo, y el estado deja de coincidir con el indicador de ocupada, que sí vuelve a libre. Como Regresando del taller rechaza los pedidos de domicilio y de traslado, los pedidos que lleguen después quedan afectados, como se explica en el cuarto hallazgo. El test con el estado aislado también falla, lo que ubica el defecto en la clase TrasladandoPacienteClinicaState y no en Ambulancia.

Hallazgo: Espera dentro de un bloque sincronizado

- **Componente:** modelo.modeloDominio.ambulancia.TrasladandoPacienteClinicaState
- **Método analizado:** public void RetornoClinica(), llamado desde public synchronized void regresarSinPaciente()
- **Clasificación del defecto:** Defecto de concurrencia / Manejo incorrecto de un recurso compartido.
- **Severidad:** Media (en cada regreso de un traslado bloquea durante un segundo a todos los hilos que usan la ambulancia).

1. Descripción del Hallazgo

Para simular el viaje de vuelta, RetornoClinica() del estado de traslado duerme un segundo (Thread.sleep(1000)). El problema es que se ejecuta dentro de regresarSinPaciente(), que es sincronizado: mientras duerme, el hilo mantiene tomada la ambulancia y ningún otro hilo puede usarla, ni siquiera para consultar si está ocupada. El test testRegresoNoBloqueaLaAmbulancia lanza el regreso en un hilo, espera a que ese hilo esté dentro del sleep y recién ahí mide cuánto tarda en responder isOcupado():

Elemento	Especificación declarada	Observación
Comportamiento esperado	Consultar la ambulancia responde de inmediato (menos de 100 ms)	Es una consulta, no modifica nada.
Código	Thread.sleep(1000) con la ambulancia tomada	La ambulancia no se libera durante la espera.
Resultado del test	isOcupado() tardó cerca de 1 segundo	La consulta quedó bloqueada.

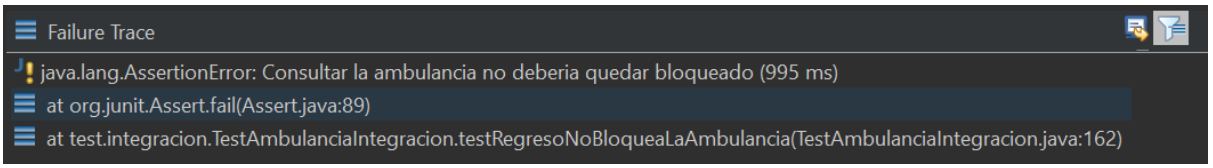


Figura 5: Falla de testRegresoNoBloqueaLaAmbulancia. La consulta tarda cerca de un segundo.

2. Análisis del Impacto en la Calidad

Con varios asociados y el operario usando la ambulancia, cada regreso de un traslado deja a todos los demás hilos esperando un segundo, incluso para una simple consulta. Además, el sleep está dentro de un bloque try con un catch vacío que descarta cualquier excepción, incluida la interrupción del hilo, así que si alguien intenta detenerlo la señal se pierde sin dejar rastro.

Hallazgo: Condición invertida en el hilo de retorno automático

- **Componente:** modelo.modeloDominio.ambulancia.RetornoAutomatico
- **Método analizado:** public void run()
- **Clasificación del defecto:** Defecto de lógica / Condición de bucle invertida.
- **Severidad:** Alta (el retorno automático nunca se ejecuta).

1. Descripción del Hallazgo

El test testRetornoAutomaticoConSimulacionActiva deja la ambulancia atendiendo un domicilio, lanza el hilo de retorno con la simulación activa y la revisa en los valores límite del intervalo de 10 segundos. A los 9 segundos sigue en el domicilio, que es lo

correcto, pero a los 11 ya tendría que estar regresando a la clínica y sigue en el domicilio. El motivo es que el bucle del hilo tiene la condición al revés: en lugar de repetirse mientras la simulación está activa, se repite mientras está terminada:

Elemento	Especificación declarada	Observación
Javadoc de run()	"Periódicamente intenta retornar la ambulancia a la clínica mientras la simulación esté activa"	Comportamiento esperado.
Código	while (!ambulancia.isSimulacionActiva())	La condición está negada.
Resultado con simulación activa	El hilo termina apenas arranca	Nunca llama a retornoAutomatico().
Resultado con simulación terminada	El hilo sigue vivo después de 1 segundo	Duerme el intervalo completo antes de salir.

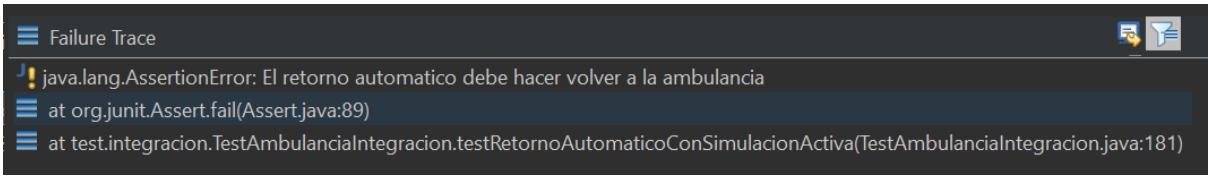


Figura 6: Falla de testRetornoAutomaticoConSimulacionActiva. A los 11 segundos la ambulancia sigue en el domicilio.

2. Análisis del Impacto en la Calidad

Con la simulación activa la condición es falsa desde el principio y el hilo termina sin hacer nada, así que la ambulancia nunca vuelve sola a la clínica. Con la simulación terminada pasa lo contrario: el hilo entra al bucle, duerme los 10 segundos y recién ahí sale, cuando tendría que terminar enseguida. Esto lo muestra testRetornoAutomaticoConSimulacionFinalizada, que espera que el hilo termine en menos de un segundo. El método retornoAutomatico() de Ambulancia funciona bien cuando se lo llama directamente (lo prueban los tests con mocks y los flujos de domicilio y mantenimiento), así que el defecto está solamente en el hilo.

Hallazgo: Pedido aceptado aunque el estado lo rechaza

- **Componente:** modelo.modeloDominio.ambulancia.Ambulancia
- **Método analizado:** public synchronized void atenderDomicilio(Asociado a); lo mismo ocurre en trasladarALaClinica() y solicitarMantenimiento()
- **Clasificación del defecto:** Defecto de lógica / Estado inconsistente entre Ambulancia y su estado.
- **Severidad:** Media (el pedido se pierde y se informa como atendido).

1. Descripción del Hallazgo

El test testPedidoRechazadoNoOcupaLaAmbulancia manda la ambulancia al taller y la hace volver, con lo que queda en Regresando del taller y libre. Después un asociado pide atención a domicilio. Ese estado no acepta el pedido y la deja como está, pero Ambulancia ya la había marcado como ocupada antes de consultar al estado, y además avisa que el asociado es atendido:

Elemento	Especificación declarada	Observación
Javadoc de RegresandoDelTallerState	"En este estado, la ambulancia no puede atender solicitudes de atención a domicilio" y el constructor la marca "como no ocupada"	Comportamiento esperado.
Código	setOcupado(true) y aviso "es atendido a domicilio" antes de delegar en el estado	Ambulancia no sabe si el estado aceptó el pedido.
Resultado del test	El estado sigue en RegresandoDelTallerState con isOcupado() en true	El pedido se rechaza pero la ambulancia queda ocupada.

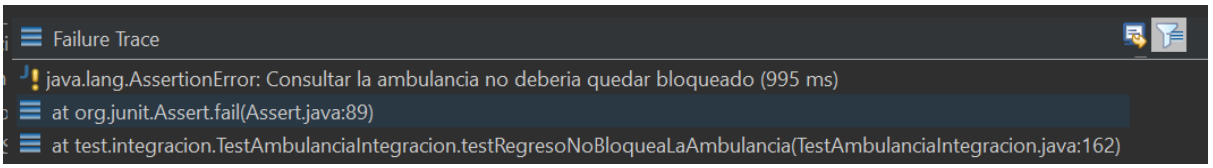


Figura 7: Falla de testPedidoRechazadoNoOcupaLaAmbulancia. La ambulancia queda ocupada aunque el pedido se rechazó.

2. Análisis del Impacto en la Calidad

En la simulación esto pasa después de cada mantenimiento, porque la ambulancia vuelve libre en Regresando del taller y, con el hilo de retorno automático fallando, nada la pasa a Disponible. Por el primer hallazgo, también pasa después de cada traslado. El asociado figura como atendido sin que la ambulancia haya ido al domicilio, y los demás pedidos esperan mientras tanto. Los métodos de State no devuelven nada, así que Ambulancia no tiene forma de enterarse de que el estado rechazó el pedido: el defecto está en cómo Ambulancia delega, no en una clase de estado en particular.

Conclusiones del testing con mocks y concurrencia

Aislar las piezas en los dos sentidos fue lo que permitió ubicar cada defecto. Los mocks de State mostraron que Ambulancia delega bien en su estado y maneja bien la espera, la ocupación y el fin de la simulación. El mock de Ambulancia mostró que cinco de los seis estados respetan la especificación y ubicó la transición incorrecta en el estado de traslado. Y con las piezas reales aparecieron los defectos que solo se ven al integrar: el regreso que bloquea la ambulancia durante un segundo, el hilo de

retorno automático que nunca actúa y los pedidos que se dan por atendidos aunque el estado los rechace.

Tres de los defectos se encadenan en la simulación: después de un traslado o de un mantenimiento la ambulancia queda en Regresando del taller, el hilo de retorno automático no la devuelve a Disponible y los pedidos siguientes se dan por atendidos sin que la ambulancia salga. Por eso consideramos que el módulo de la ambulancia no se puede usar en una simulación real hasta corregir los defectos 1, 3 y 4. El defecto 2 no rompe el funcionamiento, pero hace que la simulación sea más lenta de lo necesario.

Por último, probar código con hilos nos obligó a cuidar que los tests no dependan de la suerte. Esperar a que el otro hilo llegue efectivamente al punto que nos interesa, poner siempre un tiempo máximo y tomar los valores límite con un margen hizo que los resultados sean los mismos en cada ejecución.